

Polars 使用教程

1. 什么是 Polars?

Polars 是一个高性能的 DataFrame 库，用 Rust 编写，提供 Python 和 Rust API。它比 Pandas 更快，支持懒执行（Lazy Evaluation）和并行处理。

2. 安装

代码块

```
1 pip install polars
```

如果需要读取 Excel 文件：

代码块

```
1 pip install polars[excel]
```

3. 基础操作

3.1 导入库

代码块

```
1 import polars as pl
```

3.2 创建 DataFrame

代码块

```
1  # 从字典创建
2  df = pl.DataFrame({
3      "name": ["Alice", "Bob", "Charlie"],
4      "age": [25, 30, 35],
5      "salary": [5000, 6000, 7000]
6  })
7
8  # 从列表创建
9  df = pl.DataFrame([
10     {"name": "Alice", "age": 25},
11     {"name": "Bob", "age": 30}
12 ])
13
14 # 查看 DataFrame
15 print(df)
16 print(df.head())      # 前5行
17 print(df.tail(3))     # 后3行
18 print(df.shape)       # 行列数
19 print(df.columns)     # 列名
20 print(df.dtypes)      # 数据类型
```

3.3 读取数据

代码块

```
1  # 读取 CSV
2  df = pl.read_csv("data.csv")
3
4  # 读取 Parquet (推荐格式, 更快)
5  df = pl.read_parquet("data.parquet")
6
7  # 读取 Excel
8  df = pl.read_excel("data.xlsx")
```

```
9
10 # 读取 JSON
11 df = pl.read_json("data.json")
12
13 # 从数据库读取 (需要配合 SQLAlchemy)
14 import sqlalchemy
15 engine = sqlalchemy.create_engine("sqlite:///database.db")
16 df = pl.read_database("SELECT * FROM table_name", engine)
```

4. 数据选择与过滤

4.1 选择列

代码块

```
1 # 选择单列
2 df.select("name")
3
4 # 选择多列
5 df.select(["name", "age"])
6
7 # 使用表达式选择
8 df.select(pl.col("name"), pl.col("age"))
9
10 # 排除某些列
11 df.select(pl.exclude("salary"))
```

4.2 过滤行

代码块

```
1 # 简单过滤
2 df.filter(pl.col("age") > 25)
3
```

```
4  # 多条件过滤
5  df.filter(
6      (pl.col("age") > 25) &
7      (pl.col("salary") < 7000)
8  )
9
10 # 或条件
11 df.filter(
12     (pl.col("name") == "Alice") |
13     (pl.col("name") == "Bob")
14 )
15
16 # 使用 is_in
17 df.filter(pl.col("name").is_in(["Alice", "Bob"]))
```

5. 数据转换

5.1 添加新列

代码块

```
1  # 添加计算列
2  df.with_columns([
3      (pl.col("salary") / 1000).alias("salary_k"),
4      (pl.col("age") * 12).alias("age_months")
5  ])
6
7  # 添加常量列
8  df.with_columns([
9      pl.lit("employee").alias("role")
10 ])

```

5.2 修改列

代码块 重命名列

```
2 df.rename({"name": "full_name", "age": "years"})
3
4 # 转换数据类型
5 df.with_columns([
6     pl.col("age").cast(pl.Float64),
7     pl.col("salary").cast(pl.Int64)
8 ])
```

5.3 删除列

代码块

```
1 df.drop("salary")
2 df.drop(["salary", "age"])
```

6. 聚合与分组

6.1 基本聚合

代码块

```
1 # 单列聚合
2 df.select([
3     pl.col("salary").mean().alias("avg_salary"),
4     pl.col("salary").sum().alias("total_salary"),
5     pl.col("age").min().alias("min_age"),
6     pl.col("age").max().alias("max_age"),
7     pl.col("name").count().alias("count")
8 ])
9
10 # 对整个 DataFrame 聚合
11 df.aggregate([
12     pl.col("salary").mean(),
```

```
13     pl.col("age").std()
14 ])
```

6.2 分组聚合

代码块

```
1  # 按单列分组
2  df.groupby("department").agg([
3      pl.col("salary").mean().alias("avg_salary"),
4      pl.col("age").mean().alias("avg_age"),
5      pl.col("name").count().alias("employee_count")
6  ])
7
8  # 按多列分组
9  df.groupby(["department", "city"]).agg([
10     pl.col("salary").sum()
11 ])
12
13 # 分组后排序
14 df.groupby("department").agg([
15     pl.col("salary").mean()
16 ]).sort("salary", descending=True)
```

7. 排序

代码块

```
1  # 单列排序
2  df.sort("age")
3
4  # 降序排序
5  df.sort("age", descending=True)
6
7  # 多列排序
```

```
8 df.sort(["department", "age"], descending=[False, True])
```

8. 连接与合并

8.1 Join 操作

代码块

```
1 df1 = pl.DataFrame({
2     "id": [1, 2, 3],
3     "name": ["Alice", "Bob", "Charlie"]
4 })
5
6 df2 = pl.DataFrame({
7     "id": [1, 2, 4],
8     "salary": [5000, 6000, 7000]
9 })
10
11 # Inner Join
12 df1.join(df2, on="id", how="inner")
13
14 # Left Join
15 df1.join(df2, on="id", how="left")
16
17 # Full Outer Join
18 df1.join(df2, on="id", how="full")
19
20 # Cross Join
21 df1.join(df2, how="cross")
```

8.2 垂直堆叠

代码块

```
1 # 垂直连接 (类似 union)
2 pl.concat([df1, df2], how="vertical")
3
4 # 水平连接
5 pl.concat([df1, df2], how="horizontal")
```

9. 处理缺失值

代码块

```
1 # 检测缺失值
2 df.null_count()
3
4 # 填充缺失值
5 df.fill_null(0) # 用0填充
6 df.fill_null("unknown") # 用字符串填充
7
8 # 策略填充
9 df.fill_null(strategy="forward") # 向前填充
10 df.fill_null(strategy="backward") # 向后填充
11 df.fill_null(strategy="mean") # 用均值填充
12
13 # 删除含缺失值的行
14 df.drop_nulls()
15 df.drop_nulls(subset=["name", "age"]) # 指定列
```

10. 字符串操作

代码块

```
1 df.with_columns([
2     pl.col("name").str.to_uppercase().alias("name_upper"),
3     pl.col("name").str.to_lowercase().alias("name_lower"),
4     pl.col("name").str.len_chars().alias("name_length"),
5     pl.col("email").str.contains("@").alias("has_email"),
```



```
6     pl.col("name").str.starts_with("A").alias("starts_with_A"),
7     pl.col("text").str.replace("old", "new").alias("replaced")
8 ])
```

11. 日期时间操作

代码块

```
1  # 转换为日期类型
2  df.with_columns([
3      pl.col("date_str").str.strptime(pl.Date, "%Y-%m-%d")
4  ])
5
6  # 提取日期组件
7  df.with_columns([
8      pl.col("date").dt.year().alias("year"),
9      pl.col("date").dt.month().alias("month"),
10     pl.col("date").dt.day().alias("day"),
11     pl.col("date").dt.weekday().alias("weekday")
12 ])
13
14 # 日期计算
15 df.with_columns([
16     (pl.col("end_date") - pl.col("start_date")).dt.days().alias("duration")
17 ])
```

12. 懒执行 (Lazy Evaluation)

懒执行是 Polars 的核心特性，可以优化查询性能。

代码块

```
1  # 创建 LazyFrame
2  lf = pl.scan_csv("large_file.csv")
```

```

3
4 # 构建查询 (不会立即执行)
5 result = (
6     lf
7     .filter(pl.col("age") > 25)
8     .group_by("department")
9     .agg([
10         pl.col("salary").mean().alias("avg_salary")
11     ])
12     .sort("avg_salary", descending=True)
13     .limit(10)
14 )
15
16 # 执行查询
17 df_result = result.collect()
18
19 # 查看查询计划
20 print(result.explain())
21
22 # 优化查询计划
23 print(result.explain(optimized=True))

```

13. 窗口函数

代码块

```

1 df.with_columns([
2     # 排名
3     pl.col("salary").rank(method="dense").alias("salary_rank"),
4
5     # 组内排名
6
7     pl.col("salary").rank(method="dense").over("department").alias("dept_rank"),
8
9     # 移动平均
10    pl.col("salary").rolling_mean(window_size=3).alias("moving_avg"),
11
12    # 累计求和
13    pl.col("salary").cum_sum().alias("cumulative_salary"),
14
15    # 组内均值

```

```
15     pl.col("salary").mean().over("department").alias("dept_avg_salary"),
16
17     # 滞后值
18     pl.col("salary").shift(1).alias("prev_salary"),
19
20     # 领先值
21     pl.col("salary").shift(-1).alias("next_salary")
22 ])
```

14. 应用自定义函数

代码块

```
1  # 使用 map_elements (逐元素)
2  df.with_columns([
3      pl.col("name").map_elements(lambda x: x.upper()).alias("name_upper")
4  ])
5
6  # 使用 apply (整列)
7  df.with_columns([
8      pl.col("salary").apply(lambda x: x * 1.1).alias("salary_after_tax")
9  ])
```

15. 数据导出

代码块

```
1  # 导出为 CSV
2  df.write_csv("output.csv")
3
4  # 导出为 Parquet
5  df.write_parquet("output.parquet")
6
7  # 导出为 Excel
8  df.write_excel("output.xlsx")
```

```
9
10 # 导出为 JSON
11 df.write_json("output.json")
12
13 # 导出到数据库
14 df.write_database("table_name", engine, if_exists="replace")
```

16. 实用技巧

16.1 链式调用

代码块

```
1 result = (
2     pl.read_csv("data.csv")
3     .filter(pl.col("age") > 25)
4     .with_columns([
5         (pl.col("salary") / 1000).alias("salary_k")
6     ])
7     .group_by("department")
8     .agg([
9         pl.col("salary_k").mean().alias("avg_salary_k"),
10        pl.col("age").mean().alias("avg_age")
11    ])
12     .sort("avg_salary_k", descending=True)
13     .limit(10)
14 )
```

16.2 条件逻辑

代码块

```
1 df.with_columns([
2     pl.when(pl.col("age") < 30)
```

```
3     .then(pl.lit("young"))
4     .when(pl.col("age") < 50)
5     .then(pl.lit("middle"))
6     .otherwise(pl.lit("senior"))
7     .alias("age_group")
8 ])
```

16.3 性能优化建议

代码块

```
1  # 1. 使用懒执行处理大数据
2  lf = pl.scan_parquet("large_dataset.parquet")
3
4  # 2. 只选择需要的列
5  lf.select(["col1", "col2"])
6
7  # 3. 尽早过滤数据
8  lf.filter(pl.col("date") > "2023-01-01")
9
10 # 4. 使用 Parquet 格式 (比 CSV 快很多)
11 df.write_parquet("data.parquet")
12
13 # 5. 利用并行处理 (Polars 默认启用)
```

17. 完整示例

!!! 新版 Polars 中 `pl.date_range()` 已改为 `pl.date_ranges()` 或使用 `pl.datetime_range()`。

代码块

```
1  import polars as pl
2
3  # 创建示例数据
4  df = pl.DataFrame({
5      "employee_id": range(1, 11),
6      "name": ["Alice", "Bob", "Charlie", "David", "Eve",
7              "Frank", "Grace", "Henry", "Ivy", "Jack"],
8      "department": ["HR", "IT", "IT", "Sales", "HR",
```

```

9         "Sales", "IT", "HR", "Sales", "IT"],
10     "age": [25, 30, 35, 28, 32, 45, 29, 38, 41, 33],
11     "salary": [5000, 7000, 8000, 5500, 5200,
12               6000, 7500, 5800, 6200, 7200],
13     "hire_date": pl.date_range(
14         start="2020-01-01",
15         end="2023-01-01",
16         interval="1y",
17         eager=True
18     )[:10]
19 })
20
21 # 数据分析
22 result = (
23     df
24     .filter(pl.col("age") >= 30) # 过滤年龄>=30
25     .with_columns([
26         (pl.col("salary") / 1000).round(2).alias("salary_k"), # 薪资 (千元)
27         pl.when(pl.col("age") < 35)
28             .then(pl.lit("Young"))
29             .otherwise(pl.lit("Senior"))
30         .alias("age_category")
31     ])
32     .group_by(["department", "age_category"])
33     .agg([
34         pl.col("salary_k").mean().round(2).alias("avg_salary_k"),
35         pl.col("salary_k").sum().round(2).alias("total_salary_k"),
36         pl.col("employee_id").count().alias("employee_count"),
37         pl.col("age").mean().round(1).alias("avg_age")
38     ])
39     .sort(["department", "age_category"])
40 )
41
42 print(result)

```

18. Polars vs Pandas 对比

特性	Polars	Pandas
性能	更快 (Rust 底层)	较慢

内存效率	更高	较低
并行处理	自动并行	需要手动
懒执行	支持	不支持
API 设计	更现代化	成熟但复杂
学习曲线	较陡	平缓
生态系统	发展中	非常成熟

这个教程涵盖了 Polars 的主要功能。建议您从基础操作开始练习，逐步掌握高级特性。Polars 特别适合处理大规模数据集，性能优势明显！

清除 pip 缓存

pip cache purge

清华大学镜像

pip install polars -i <https://pypi.tuna.tsinghua.edu.cn/simple>

```
管理员: Anaconda Powe × + ▾
(base) PS C:\Users\Administrator> pip install polars
Collecting polars
  Downloading polars-1.40.1-py3-none-any.whl.metadata (10 kB)
Collecting polars-runtime-32==1.40.1 (from polars)
  Downloading polars_runtime_32-1.40.1-cp310-abi3-win_amd64.whl.metadata (1.5 kB)
Downloading polars-1.40.1-py3-none-any.whl (828 kB)
 828.7/828.7 kB 20.0 kB/s eta 0:00:00
Downloading polars_runtime_32-1.40.1-cp310-abi3-win_amd64.whl (51.8 MB)
 6.8/51.8 MB 25.0 kB/s eta 0:30:02
ERROR: Operation cancelled by user
(base) PS C:\Users\Administrator> pip cache dir
c:\users\administrator\appdata\local\pip\cache
(base) PS C:\Users\Administrator> pip show polars
WARNING: Package(s) not found: polars
(base) PS C:\Users\Administrator> pip cache purge
Files removed: 784
(base) PS C:\Users\Administrator> pip install polars -i https://pypi.tuna.tsinghua.edu.cn/simple
Looking in indexes: https://pypi.tuna.tsinghua.edu.cn/simple
Collecting polars
  Downloading https://pypi.tuna.tsinghua.edu.cn/packages/ea/91/74fc60d94488685a92ac9d49d7ec55f3e91fe9b77942a6235a5fa7f249c3/polars-1.40.1-py3-none-any.whl (828 kB)
 828.7/828.7 kB 25.6 MB/s eta 0:00:00
Collecting polars-runtime-32==1.40.1 (from polars)
  Downloading https://pypi.tuna.tsinghua.edu.cn/packages/06/d9/d091d8fb5cbed5e9536adfed955c4c89987a4cc3b8e73ae4532402b91c74/polars_runtime_32-1.40.1-cp310-abi3-win_amd64.whl (51.8 MB)
 51.8/51.8 MB 29.8 MB/s eta 0:00:00
Installing collected packages: polars-runtime-32, polars
Successfully installed polars-1.40.1 polars-runtime-32-1.40.1
(base) PS C:\Users\Administrator> |
```



polars-1.40.1.dist-info

D:\anaconda3\Lib\site-packages

修改日期: 2026/5/10 23:29



polars

D:\anaconda3\Lib\site-packages

修改日期: 2026/5/10 23:29



polars_runtime_32-1.40.1.dist-info

D:\anaconda3\Lib\site-packages

修改日期: 2026/5/10 23:29



_polars_runtime_32

D:\anaconda3\Lib\site-packages

修改日期: 2026/5/10 23:29

阿里云镜像

pip install polars -i <https://mirrors.aliyun.com/pypi/simple/>

豆瓣镜像

pip install polars -i <https://pypi.douban.com/simple/>

修改说明:

导入 date: 添加了 from datetime import date

手动创建日期列表: 使用列表推导式生成 10 个日期 (2020-01-01 到 2029-01-01)

替换 pl.date_range(): 直接用 hire_dates 列表赋值给 hire_date 列

这个方案更加稳定, 不依赖 Polars 的日期范围函数, 避免了版本兼容性问题。现在可以重新运行代码了!

代码块

```
1 import polars as pl
2 from datetime import date
3
4 # 创建示例数据
5 hire_dates = [date(2020 + i, 1, 1) for i in range(10)]
6
7 df = pl.DataFrame({
8     "employee_id": range(1, 11),
9     "name": ["Alice", "Bob", "Charlie", "David", "Eve",
```



```
10         "Frank", "Grace", "Henry", "Ivy", "Jack"],
11     "department": ["HR", "IT", "IT", "Sales", "HR",
12                    "Sales", "IT", "HR", "Sales", "IT"],
13     "age": [25, 30, 35, 28, 32, 45, 29, 38, 41, 33],
14     "salary": [5000, 7000, 8000, 5500, 5200,
15               6000, 7500, 5800, 6200, 7200],
16     "hire_date": hire_dates
17 })
18
19 # 数据分析
20 result = (
21     df
22     .filter(pl.col("age") >= 30) # 过滤年龄>=30
23     .with_columns([
24         (pl.col("salary") / 1000).round(2).alias("salary_k"), # 薪资 (千元)
25         pl.when(pl.col("age") < 35)
26         .then(pl.lit("Young"))
27         .otherwise(pl.lit("Senior"))
28         .alias("age_category")
29     ])
30     .group_by(["department", "age_category"])
31     .agg([
32         pl.col("salary_k").mean().round(2).alias("avg_salary_k"),
33         pl.col("salary_k").sum().round(2).alias("total_salary_k"),
34         pl.col("employee_id").count().alias("employee_count"),
35         pl.col("age").mean().round(1).alias("avg_age")
36     ])
37     .sort(["department", "age_category"])
38 )
39
40 print(result)
```